

アクター指向言語における 「1 アクター = 1 実行担体」モデルの対応表

— Erlang の「プロセス」・OS スレッド・軽量プロセスの区別と、AIPL/ABCL^c+ の位置づけ —

ABCL^c+ / AIOS 処理系 (yaskodama/abclcp)

2026 年 07 月 31 日

概要

「Erlang などのアクター言語に、アクター 1 つにつきプロセス 1 つに限定したモデルはあるか」という問いに答える。結論を先に書くと、**ある**。ただし「プロセス」の語が指すものを 3 層に分けないと問いが定まらない。Erlang は確かに「1 アクター = 1 プロセス」だが、そのプロセスは OS のプロセスでもスレッドでもなく BEAM の軽量プロセスである。一方、**1 アクターを 1 本の OS スレッドに固定する処理系も、1 アクターを 1 個の OS プロセスに固定する処理系も実在する**。前者の代表は Celluloid・Pykka・AmbientTalk・本処理系 AIPL、後者の代表は Ray と Thespian である。Akka と CAF は既定では多重化しつつ、**オプティンで 1 対 1 に切り替えられる**。

本書は主要なアクター指向言語・処理系を横断して、(1) アクターが載る実行担体、(2) 1 対 1 に専有するか、(3) アクター本体がブロックできるか、(4) 実用上のアクター数、の 4 点で対応表を作る (§3)。そのうえで、なぜ 1 対 1 の OS スレッド束縛が稀なのか (コスト、§5)、それでもなぜ選ぶのか (同期待ちが書けること、§6) を述べ、最後に AIPL の設計上の位置づけを示す (§7)。

AIPL は 1 アクターにつき 1 本の OS スレッドを生成する。本書のために実測したところ、**4000 アクターまでは生成でき、5000 で Thread.create: Resource temporarily unavailable で失敗した**。上限は macOS の `kern.num_taskthreads = 4096` である。この数字がこのモデルの代償を端的に表している。

目次

1	問いの整理 — 「プロセス」は 3 層ある	2
2	対応表の読み方	2
3	対応表	4
3.1	アクターが言語の構文である言語	4
3.2	ライブラリ・フレームワーク	6
3.3	参考：アクターではないが対比される仕組み	7
4	「1 アクター = 1 OS 実行資源」を採る処理系 (抽出)	9
5	なぜ稀か — コスト	9
6	ではなぜ選ぶのか — ブロックできること	10

7	AIPL / ABCL ^{c+} の位置づけ	11
7.1	何をしているか	11
7.2	なぜこの設計なのか	11
7.3	代償が言語仕様に現れている	11
7.4	他との比較でどこに立つか	12
8	まとめ	12
9	出典	12

1 問いの整理 — 「プロセス」は 3 層ある

アクター言語の文献で「プロセス」と呼ばれるものには、コストが 3 桁以上違う 3 つの層がある。この区別をしないと「1 アクター 1 プロセス」という言明は意味を持たない。

層	何か	1 個あたりの費用	実用上の個数
OS プロセス	独立したアドレス空間。カーネルが管理	数 MB ～、生成 ms オーダー	10 ²
OS スレッド (pthread)	アドレス空間を共有。カーネルが管理・プリエンプト	スタック 512 KB (macOS) ～ 8 MB (Linux)	10 ³
軽量プロセス / グリーンスレッド	言語ランタイムが管理する私有スタック。M:N で OS スレッドへ載せる	BEAM で 326 語 (≈2.6 KB)	10 ⁶
ターン (スタックを持たない)	「1 メッセージ = 1 関数呼び出し」。実行の間だけプールのスレッドを借りる	Pony のアクターで ≈240 B	10 ^{6~7}

Erlang の「プロセス」は 3 段目である。 Erlang は確かに「1 アクター = 1 プロセス」を厳密に守るが、それは OS から見れば何でもない。BEAM はコア数と同じだけのスケジューラスレッドを立て、その上に 10⁶ 個の軽量プロセスを M:N で載せる。したがって「Erlang はアクターごとにプロセスを持つから重いのでは」という直感は誤りで、むしろ軽いからこそ 1 対 1 を貫ける。

問いを言い直すと、次の 2 つになる。

1. OS の実行資源 (スレッド / プロセス) を 1 アクターに専有させる処理系はあるか。→ ある (§4)。
2. それを言語の既定にしている処理系はあるか。→ ごく少数だが、ある。AIPL はその 1 つである。

2 対応表の読み方

対応表では次の 4 軸で分類する。

実行担体 アクターが実行されるとき、何の上で走るか。OS プロセス / OS スレッド / 軽量プロセス / プールのターン。

1 対 1 か その担体をアクターが専有するか、多数のアクターで多重化するか。○ は専有、× は多重化。

ブロック可 アクターのメソッド本体の途中で同期的に待てるか。待つには私有スタックが要る。待てない処理系 (run-to-completion) では、同期呼び出しは継続・コールバック・await に書き換える必要がある。

実用規模 1 ノードで無理なく作れるアクター数の桁。

この 4 軸を選んだのは、**第 3 軸が他の 3 つを決めてしまう**からである。アクター本体の途中で待てるようにすると、待っている間の呼び出し状態を保持する場所が要る。その場所として OS スレッドのスタックを使えば第 1 軸と第 2 軸が「OS スレッド・専有」に決まり、その結果として第 4 軸がカーネルの上限で頭打ちになる。逆に待てない設計 (run-to-completion) にすれば担体を専有する必要がなくなり、アクターは単なるオブジェクトになって 10^6 個作れる。つまりこの表は、実質的には「**同期的に待てるか**」という 1 つの設計判断がどこまで波及するかの一覧である。この点は §6 で改めて述べる。

なお「メールボックスの選択受信ができるか」「監視ツリーがあるか」「メッセージ順序をどこまで保証するか」といった軸もありうるが、これらは担体の選択とは概ね独立に決められるので、本表では扱わない (AIPL と Erlang の比較としては §7 の末尾に挙げた)。

凡例：[†] を付けた行は、標準的な文献・処理系の知識にもとづく記述で、本書の作成にあたって一次資料での再確認をしていないものである。[†] の無い行は §9 に挙げた資料で確認した。AIPL の行は実装のソースと実測による。

3 対応表

3.1 アクターが言語の構文である言語

言語（初出）	実行担体	1 対 1 か	ブロック可	実用規模	備考
Act1 / Act2 / Act3 (1981-)†	概念上のみ。実装は Lisp 上のスケジューラ	—	—	—	理論的なアクターモデルの原型。担体の議論をしていない
ABCL/1 (1986)†	「各オブジェクトが固有の活動を持つ」。実装は Lisp の軽量プロセス	概念上 ○	○ (now)	10 ³	past/now/future の 3 送信。AIPL の直接の祖先
Concurrent Smalltalk (1987)†	処理系の軽量プロセス	×	○	10 ⁴	
Erlang (1986/98)	BEAM 軽量プロセス 。コア数個のスケジューラスレッドへ M:N	○ (軽量プロセスと)	○ (receive)	10 ⁶	新規プロセスは 326 語。既定の上限 +P 262144。選択的受信あり。 問いの「1 アクター 1 プロセス」はここでは成立するが、プロセス = 軽量プロセス
Elixir / Gleam / LFE†	同上 (BEAM を共有)	○	○	10 ⁶	実行モデルは Erlang と同一
E (1997)	vat = 1 OS スレッド。1 つの vat に多数のオブジェクトが載る	×	×	—	1 vat に N 個のオブジェクトが載るので、担体はアクター単位ではない。非同期送信のみで、同期呼び出しと構文で区別する
AmbientTalk (2006)	アクター = イベントループ = 1 JVM スレッド	○	×	10 ^{2~3}	E の vat モデルを継承。 1 アクター 1 OS スレッドの言語レベルの例

言語 (初出)	実行担体	1 対 1 か	ブロック可	実用規模	備考
SALSA (1999)	初期は 1 アクター 1 Java スレッド。SALSA Lite でステージ (スレッド) に多数のアクターを載せる方式へ	初期 ○ → ×	○ [†]	10 ³ → 10 ⁵	同一言語がスレッド専有から多重化へ移った実例
Scala Actors (2006, 2.11 で非推奨)	receive = スレッドを占有 / react = 継続クロージャのみ	選べる	receive なら ○	react で 10 ⁵	同じ言語の中に両モデルが並置された唯一に近い例。Haller & Odersky「Actors that Unify Threads and Events」
Io (2002) [†]	コルーチン (言語ランタイムが管理)	○ (コルーチンと)	○	10 ⁴	オブジェクトが動的にアクター化する
Pony (2015)	担体を持たない。behaviour は run-to-completion でコア数個のスケジューラが work-stealing	×	×	10 ^{6~7}	ブロックを設計として禁じている。空アクター ≈240 B。参照能力型でデータ競合が起こらないことを保証
Encore (2015) [†]	Pony ランタイム上の active object	×	× (future)	10 ⁶	能力型 + 並列コンビネータ
Swift actor (2021)	Executor 上の Job。既定は協調スレッドプール (コア数程度)	×	× (await)	10 ⁶	カスタム executor でスレッド固定は可能だが既定ではない。再入 (reentrancy) を許す
Motoko (2019)	canister = アクター。1 メッセージずつ処理	○ (canister)	× (await)	—	await がメッセージを分割し commit point になる。待ちが状態のコミット境界になるという珍しい意味論
Rebeca (2001) [†]	モデル検査用。各 rebec が固有の制御を持つ	概念上 ○	—	—	実行より検証が目的。Timed Rebeca は時間も持つ

言語（初出）	実行担体	1 対 1 か	ブロック可	実用規模	備考
D std.concurrency (2010) [†]	spawn が OS スレッドを作る	○	○ (receive)	10 ³	標準ライブラリだが言語と密着。1 アクター 1 OS スレッドの例
AIPL / ABCL ^{c+} (2025–26)	OS スレッド (pthread)。new ごとに Thread.create	○ (既定かつ唯一)	○ (now)	実測 4096	OCaml 5.1 の Thread。同一ドメイン内なので並行だが並列ではない。§7

3.2 ライブラリ・フレームワーク

処理系（言語）	実行担体	1 対 1 か	ブロック可	実用規模	備考
Akka / Akka.NET (JVM/.NET)	既定は共有スレッドプール (dispatcher、8–64 スレッド)。PinnedDispatcher でアクター専用スレッド	既定 × / 可 ○	× (ask)	10 ⁶	run-to-completion なので待てない。同期呼び出しは ask が返す Future で書く。「重要なアクターに専用スレッドを与える」「バルクヘッドを作る」ための機能として 1 対 1 が用意されている
CAF (C++)	既定は協調スケジューリング (数百バイト)。spawn<detached> で専有スレッド	既定 × / 可 ○	detached なら ○	10 ⁶	ブロッキング I/O を呼ぶアクターを他から隔離する用途。「ブロックしたいなら専有スレッド」という対応関係が明示的
Orleans (.NET) [†]	仮想アクター (grain)。ターン単位でスレッドプール上に活性化	×	× (await)	10 ⁶	使われていない grain は非活性化される。担体を持たないので当然
Proto.Actor / Dapr Actors [†]	同上 (ターンベース)	×	×	10 ⁶	

処理系（言語）	実行担体	1 対 1 か	ブロック可	実用規模	備考
Actix (Rust) [†]	Arbiter (=1 スレッド) に多数のアクター。専用 Arbiter を与えれば 1 対 1	既定 × / 可 ○	×	10 ⁶	
Ractor / Bastion (Rust) [†]	tokio タスク／ランタイムの軽量プロセス	×	×	10 ⁶	Bastion は Erlang の監視ツリーを模す
Cloud Haskell [†]	forkIO 軽量スレッド (HEC へ M:N)	○ (軽量スレッドと)	○	10 ⁶	Erlang に最も近い構図
Ray (Python)	1 アクター = 1 専有 OS プロセス (ワーカー)	○ (OS プロセス)	○	10 ^{2~3} / ノード	タスクと違いワーカーを再利用しない。問いに対する最も直接的な「はい」
Thespian (Python)	multiprocTCPBase 等では 1 アクター 1 OS プロセス。simpleSystemBase は単一プロセス	選べる	○	10 ²	同一のアクターコードのまま担体だけ差し替えられる設計
Pykka (Python)	ThreadingActor = 1 threading.Thread	○	○	10 ³	「各 ThreadingActor は通常のスレッドで実行される」
Celluloid (Ruby)	1 アクター = 1 Ruby スレッド。メソッド呼び出しごとに Fiber で待ちを退避	○	○	10 ³	「2000 個以上のアクターが作れない」という報告が実際にある。このモデルの代償がそのまま現れた例
Nact / Comedy (JS) [†]	単一のイベントループ	×	×	10 ⁵	

3.3 参考：アクターではないが対比される仕組み

		実行担体	1 対 1 か	ブロック可	実用規模	備考
Go	(goroutine + channel) [†]	goroutine (初期スタック 2KB、M:N)	慣用的に ○	○	10 ⁶	CSP であってアクターではない (宛先が名前ではなくチャンネル)。「1 状態 = 1 goroutine」の書き方が事実上アクター
Vert.x	(JVM) [†]	verticle をイベントループスレッドに固定。1 loop : N verticle	×	×	10 ⁵	
Java	21 virtual threads [†]	JVM 管理の軽量スレッド	—	○	10 ⁶	「1 アクター 1 スレッド」の経済性を変える。§6

4 「1 アクター = 1 OS 実行資源」を採る処理系（抽出）

対応表から該当するものだけを抜き出すと、次のようになる。

	担体	処理系
既定でそう	OS プロセス OS スレッド	Ray (Python)、Thespian (multiproc 系) AIPL/ABCL^c+ 、AmbientTalk、Celluloid (Ruby)、Pykka ThreadingActor、D std.concurrency [†] 、初期の SALSA
オプトインで	OS スレッド	Akka PinnedDispatcher、CAF spawn<detached>、Actix の専用 Arbiter [†] 、旧 Scala Actors の receive、Swift のカスタム executor
軽量プロセスと 1 対 1	言語ランタイム	Erlang / Elixir / Gleam / LFE、Cloud Haskell [†] 、Io [†]
担体を専有しない	プールのターン	Pony、Encore、Swift (既定)、Akka (既定)、CAF (既定)、Orleans、Proto.Actor、Dapr、Nact

読み取れることは 3 つある。

1. OS スレッド専有は、言語の既定としては少数派である。該当するのは研究用言語（AmbientTalk、初期 SALSA、AIPL）と、動的言語のライブラリ（Celluloid、Pykka）に偏る。
2. 大規模実用系はすべてオプトインにしている。Akka と CAF は、既定では多重化しつつ「ブロッキング I/O を呼ぶアクターだけ隔離する」ために専有スレッドを用意している。つまり専有は例外的な道具という位置づけである。
3. 分散フレームワークでは 1 アクター 1 OS プロセスが主流になる。Ray と Thespian がそうで、これは並列性（GIL の回避）と障害隔離のためであって、アクターモデルの要請ではない。

5 なぜ稀か — コスト

担体	1 個あたり	出典・備考
BEAM 軽量プロセス	326 語（初期ヒープ 233 語を含む）	Erlang 効率化ガイド
Pony のアクター goroutine	≈240 B（64bit の空アクター） 初期スタック 2 KB [†]	Pony FAQ
OS スレッド	スタック 512 KB（macOS）～ 8 MB（Linux）、加えてカーネル側の資源	
OS プロセス	数 MB ～	

差は容量よりもカーネルが管理する資源には上限があることに現れる。本書のために AIPL で実測した結果を挙げる。

アクター数	生成の所要時間（プロセス起動込み）	結果
1 000	0.32 秒	成功
2 000	1.03 秒	成功
3 000	2.14 秒	成功
4 000	3.66 秒	成功
5 000	—	Thread.create: Resource temporarily unavailable

```
$ sysctl kern.num_taskthreads
kern.num_taskthreads: 4096
```

上限は明らかに OS のプロセスあたりスレッド数上限である。すなわちこのモデルではアクター数の上限が言語ではなくカーネルで決まる。所要時間も超線形に伸びている（1000 個で 0.32 秒、4000 個で 3.66 秒）。

同じ現象は他の処理系でも観測されていて、Celluloid には「自分のマシンでは 2000 個以上のアクター／スレッドを作れない」という報告がそのまま issue として残っている。Erlang が既定で $2^{18} = 262\,144$ プロセスを許し、実測で 10^6 を扱えるのとは 2 桁以上違う。

6 ではなぜ選ぶのか — ブロックできること

コストが 2 桁以上不利なのに 1 アクター 1 スレッドを選ぶ理由は 1 つである。アクター本体の途中で同期的に待てるようにするためである。

メソッドの途中で待つには、待っている間の呼び出し状態をどこかに置かねばならない。置き場所は 3 つしかない。

1. **OS スレッドのスタック** — 何も書き換えずに待てる。代わりにアクター数が 10^3 で頭打ちになる。AIPL、Celluloid、Pykka、CAF の detached、Akka の PinnedDispatcher。
2. **言語ランタイムの軽量プロセスのスタック** — 待てて、かつ安い。ただしランタイムを自前で持つ必要がある。Erlang、Go、Java 21 の virtual thread。
3. **継続（クロージャ）** — スタックを持たない。代わりに待つ構文を書けないので、await やコールバックへの書き換えが必要になる。Pony、Swift、Akka、Orleans、Motoko。

この三択が同じ言語の中に並置された例が、旧 Scala Actors の receive（スレッドを占有して待つ）と react（継続だけ残して待つ）である。Haller と Odersky の「Actors that Unify Threads and Events」はまさにこの選択を統一しようとした仕事で、「アクターはフルのスレッドスタックで中断することも、継続クロージャだけで中断することもできる」と述べている。第 1 の形がスレッドベース、第 2 の形がイベントベースである。

CAF の detached も同じ対応関係を明示している。「ブロッキングなシステムコールを使うアクターはスレッドを止めて不均衡や飢餓を招くので、協調スケジューリングから外す」という説明で、**ブロックしたいなら専有スレッド**という交換条件がそのまま書かれている。

Java 21 の virtual thread は、この経済性を変えつつある。 選択肢 2 が JVM の標準機能として使えるようになったので、「待てる」とことと「安い」ことが両立する。同じことは Go では以前から成り立っていた。OS スレッド専有が正当化される場面は、今後は「ランタイムを持たない／持てない言語」に限られていく可能性が高い。

7 AIPL / ABCL^{c+} の位置づけ

7.1 何をしているか

AIPL は `new C(...)` のたびに `Thread.create` を呼び、アクターごとに 1 本の OCaml スレッドを立てる (`src/eval_thread.ml`)。アクターループはメールボックスからメッセージを取り出して逐次処理し、`now` での待ちは条件変数でブロックする。

```
ignore (Thread.create (fun () -> actor_loop actor_inst) ());
```

正確には次の 2 点に注意が要る。

- OCaml 5.1 の `Thread` は**本物の OS スレッド**である。したがってアクター数の上限は OS のスレッド数上限になる (実測 4096)。
- ただし本処理系は `Domain` を使っていないので、すべてのスレッドが単一ドメインに属する。OCaml のランタイムロックにより、**同時に OCaml コードを実行できるのは 1 本だけ**である。すなわち AIPL のアクターは**並行 (concurrent) だが並列 (parallel) ではない**。コストは OS スレッドのぶん払っているが、マルチコアの利得は現状では得ていない。

7.2 なぜこの設計なのか

AIPL は ABCL/1 の `now` (同期送信) を言語の中心に据えている。

```
var left = now stock.take(k) timeout 100 else 0;  
if (left > 0) { reply("ok, left=" ++ left); } else { reply("sold out"); }
```

これは**メソッド本体の途中で待つ**構文である。§6 の三択でいえば選択肢 1 を採ったことになり、だから 1 アクター 1 OS スレッドになる。Pony や Swift のように継続へ分割する方式を採れば安くなるが、そのときは `now` という構文自体が書けなくなる (`await` と継続に置き換わる)。

7.3 代償が言語仕様に現れている

このモデルの代償は、AIPL では 2 か所に見える形で現れている。

1. **期限付きの待ちを導入せざるを得なかった**。アクターが 1 台 1 スレッドで、待ちがそのスレッドをブロックする以上、返信が来なければそのアクターは永久に停止する。そこで `now ... timeout n else e` を入れ、待ちに上限を置いた。期限付きの待ちだけを使うプログラムではデッドロック自由が定理として言える。これは**実行モデルの制約が型システムの設計を決めた例**である。
2. **`now self.m()` を文法で禁止した**。自分自身への同期送信は、自分がそのメソッドを処理中である以上返信を作る主体がいないので必ず自己デッドロックになる。1 アクター 1 スレッドだからこそ確実にそうなるので、構文として書けなくした。

7.4 他との比較でどこに立つか

	AIPL	対照
担体	OS スレッド専有	Erlang: 軽量プロセス専有
待てるか	○ (now)	Pony/Swift: ×
上限	4096 (カーネル)	Erlang: 10^6
並列性	× (単一ドメイン)	Erlang/Pony: ○ (コア数)
選択的受信	○ (select)	Akka: × (stash で代用)
待ちの期限	○ (言語機能)	多くは API レベル (ask のタイムアウト)

対応表の中で AIPL に最も近いのは **AmbientTalk** (1 アクター 1 JVM スレッド) と **Celluloid** (1 アクター 1 Ruby スレッド) だが、両者と違って AIPL は同期待ちを言語の一級の構文として持ち、かつその期限を型検査で見える。「1 アクター 1 OS スレッド」+「同期待ちが書ける」+「待ちの期限が静的に検査される」という組み合わせは、調べた範囲では他に見当たらない。

8 まとめ

1. 「1 アクター = 1 プロセス」は Erlang で成立するが、そのプロセスは BEAM の軽量プロセスであって OS の資源ではない。
2. OS の実行資源を 1 アクターに専有させるモデルは実在する。OS プロセス単位では Ray と Thes-pian、OS スレッド単位では AmbientTalk・Celluloid・Pykka・初期 SALSA・D std.concurrency・AIPL。Akka と CAF はオプティンとして持っている。
3. 言語の既定としてそうしている例は少数で、研究用言語と動的言語のライブラリに偏る。理由は単純にコストで、アクター数の上限がカーネルで決まってしまう (AIPL の実測で 4096)。
4. それでも選ぶ理由は「メソッド本体の途中で同期的に待てる」ことに尽きる。待ち状態の置き場所がスレッドスタック／軽量プロセス／継続の三択であり、ランタイムを持たない処理系は第 1 の選択肢しか採れない。
5. AIPL はこの第 1 の選択肢を採った。その代償が「待ちに期限を書かせる」という言語仕様として表に出ている。

9 出典

本文中で確認に用いた資料。

- Erlang System Documentation, *Efficiency Guide — Processes* (初期ヒープ 233 語、新規プロセス 326 語)
https://www.erlang.org/doc/system/eff_guide_processes.html
- Ray Documentation, *Actors* (各アクターが専有 Python プロセスで動く)
<https://docs.ray.io/en/latest/ray-core/actors.html>
- Akka Documentation, *Dispatchers* (PinnedDispatcher はアクターごとに 1 スレッドのプールを持つ)
<https://doc.akka.io/libraries/akka-core/current/typed/dispatchers.html>
- CAF Documentation, *Actors* (協調スケジューリングと detached)
<https://actor-framework.readthedocs.io/en/stable/core/Actors.html>
- P. Haller, M. Odersky, *Actors that Unify Threads and Events*, 2007 (receive と react、スレッドスタックと継続クロージャ)
<https://infoscience.epfl.ch/bitstreams/1b9cd67f-c907-4fac-8cfd-b0829b000dcf/download>

- Swift Concurrency (Executor と協調スレッドプール、再入)
<https://wwdcnotes.com/documentation/wwdc21-10254-swift-concurrency-behind-the-scenes/>
- Internet Computer Documentation, *Actors and async data* (canister = アクター、await が commit point)
<https://internetcomputer.org/docs/motoko/main/writing-motoko/actors-async>
- AmbientTalk Tutorial, *Concurrent Programming with Actors* (E の vat モデル、イベントループスレッド)
<https://soft.vub.ac.be/amop/at/tutorial/actors>
- Pony FAQ, *Runtime* (スケジューラはコア数、空アクター ≈240–256 B)
<https://www.ponylang.io/faq/runtime/>
- Pykka Documentation, *Threading* (ThreadingActor は `threading.Thread` で実行される)
<https://pykka.readthedocs.io/en/release-2.x/runtimes/threading/>
- Thespian Documentation, *Using Thespian* (system base によるプロセス／スレッドの違い)
<https://thespianpy.com/doc/using.html>
- Celluloid (各アクターは専用スレッド、メソッド呼び出しは Fiber)
<https://celluloid.io/> / <https://github.com/celluloid/celluloid/issues/116> (2000 個で頭打ちの報告)
- T. Desell, C. Varela, *SALSA Lite: A Hash-Based Actor Runtime for Efficient Local Concurrency*, 2014 (ステージ方式への移行)
https://link.springer.com/chapter/10.1007/978-3-662-44471-9_7
- AIPL: 本リポジトリ `src/eval_thread.ml` (`Thread.create`)、および本書のための実測 (macOS 26.5.2, OCaml 5.1.1, `kern.num_taskthreads = 4096`)